

QUADRO 7: A EVOLUÇÃO DO DEEP LEARNING (MLP ao N-HITS)

AS BASES & MODELOS CLÁSSICOS

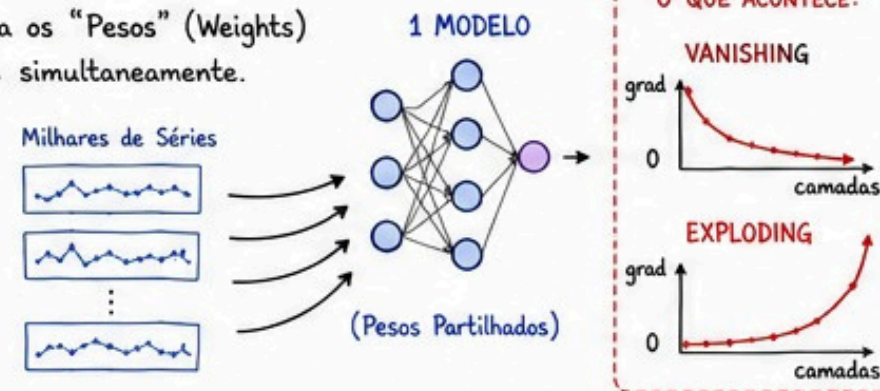
1 O PARADIGMA GLOBAL (CROSS-LEARNING)



A REGRA: 1 único modelo partilha os "Pesos" (Weights) treinando em milhares de séries simultaneamente.

PERIGO EXTREMO (SEM SCALING):

Ocorrem Vanishing / Exploding Gradients. A rede colapsa e os pesos não atualizam.



2 A ARTE DO SCALING (NEURALFORECAST)



local_scaler_type:

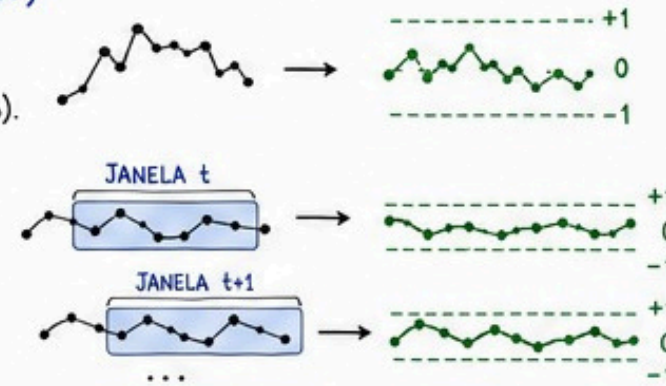
Normaliza a série INTEIRA a priori (estático).



scaler_type:

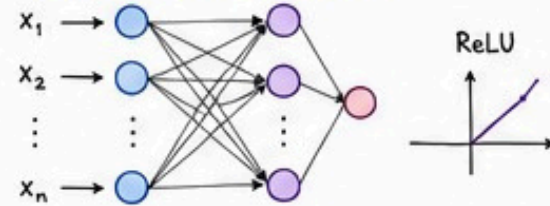
Normaliza cada Janela/Batch dinamicamente durante o treino iterativo.

(★ MELHOR PERFORMANCE!)



3 OS CLÁSSICOS (AS FUNDAÇÕES)

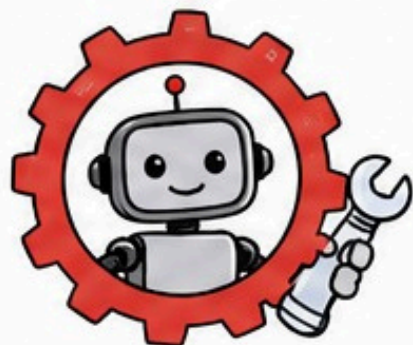
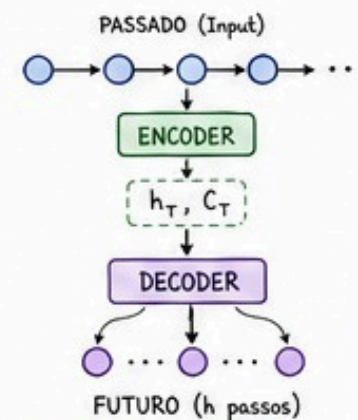
MLP (Perceptron Simples)



LSTM (ENCODER-DECODER)

ENCODER: Lê o passado e comprime a memória nos vetores Hidden State (h_T) e Cell State (C_T).

DECODER: Pega nessa "memória comprimida" e projeta o vetor do futuro (h).



CAIXA DE ALERTA GIGANTE

HYPERPARAMETER TUNING (OS MODELOS AUTO)

O desempenho em Deep Learning é 100% dependente da afinação de hiperparâmetros!



A FERRAMENTA: Modelos Auto (ex: AutoNHITS) acionam o Ray Tune ou Optuna para testar configurações.



REGRA DE SOBREVIVÊNCIA:

O parâmetro `num_samples` (quantas combinações o motor vai explorar) tem de ser OBRIGATORIAMENTE > 20 para resultados decentes!



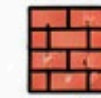
OPTUNA

Se `num_samples` ≤ 20
= Procura Preguiçosa.
Risco Alto de Ficar
num Ótimo Local!



A REVOLUÇÃO RESIDUAL

1 N-BEATS (A DESCONSTRUÇÃO DO ERRO)



Arquitetura em Blocos Empilhados.



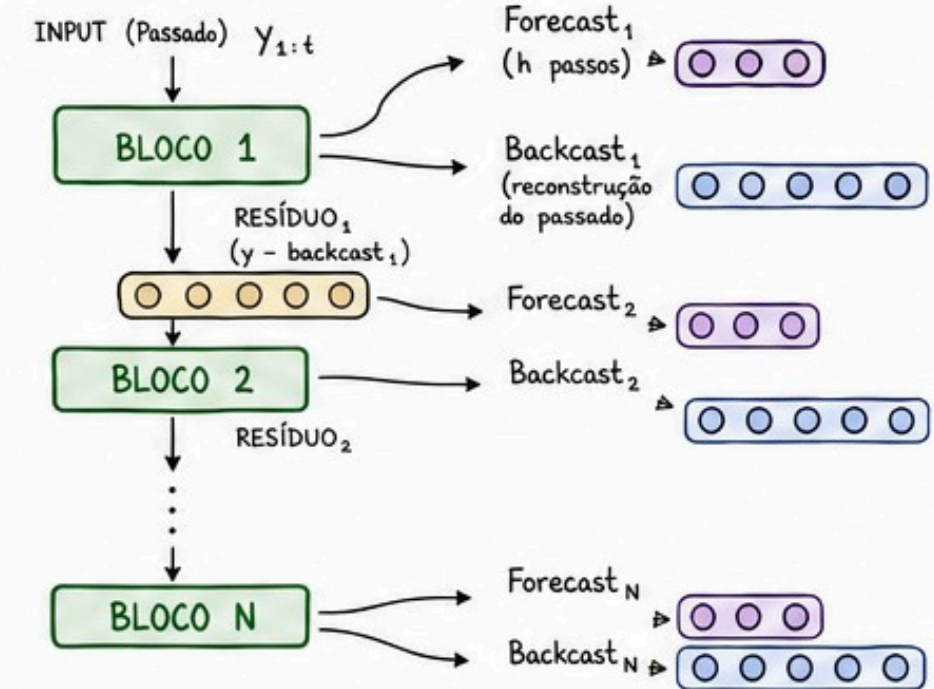
DUPLA SAÍDA (O SEGREDO):

- Forecast: O que projeta para o futuro.
- Backcast: A tentativa de reconstruir o passado.



RESIDUAL LEARNING:

O Bloco 2 não tenta prever tudo de novo. Tenta prever apenas a "fração de erro" (o resíduo) que o Bloco 1 falhou em captar!



2 N-HITS (A EVOLUÇÃO PARA O LONGO PRAZO)



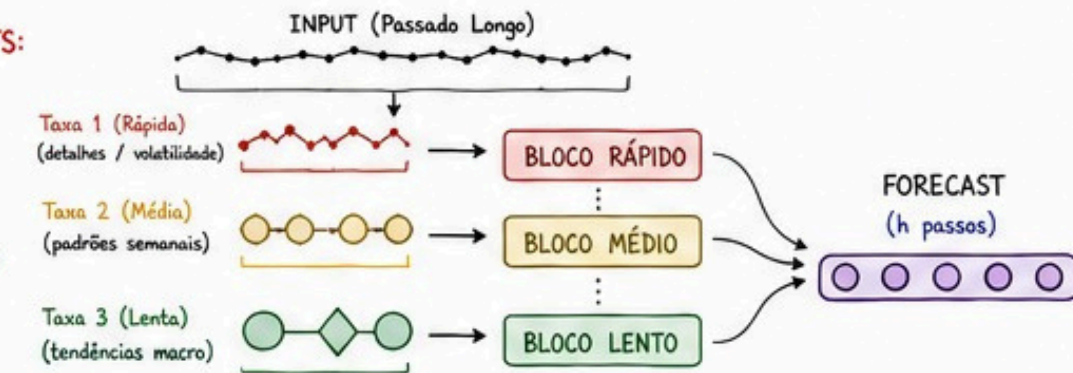
O PROBLEMA DO N-BEATS:

Muito pesado para horizontes de previsão longos (LSTF).



A SOLUÇÃO (N-HITS): MULTI-RATE INPUT POOLING

Lê a série temporal em FREQUÊNCIAS DIFERENTES (como um rádio).



RESULTADO: Muito mais LEVE, RÁPIDO e PRECISO no longo prazo!

QUADRO 8: O CÉREBRO TRANSFORMER & O ESTADO DA ARTE

O MOTOR TRANSFORMER (A ATENÇÃO)

1 SELF-ATTENTION (O NÚCLEO)



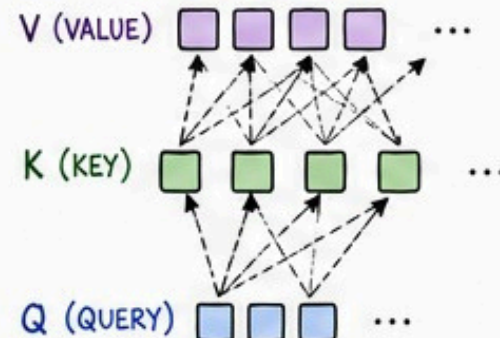
Q (Query):
O que estou
a procurar?



K (Key):
O que eu
tenho?



V (Value):
O conteúdo real.



A EQUAÇÃO MÁGICA:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{m}}\right) \times V$$

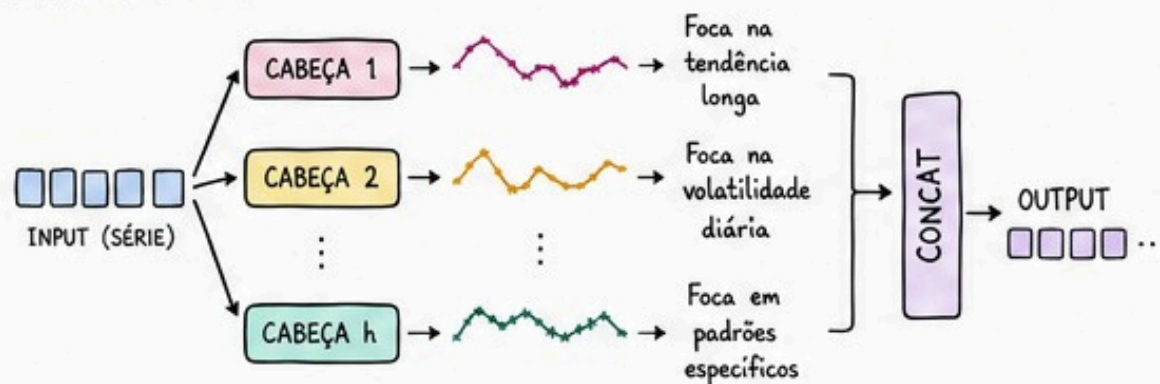
Porquê dividir por $\sqrt{m}$?

Para evitar que números gigantes congelem o Softmax (trava os Vanishing Gradients).

2 MULTI-HEAD ATTENTION



Múltiplas "cabeças" a ler a série ao mesmo tempo em paralelo.



3 O GRANDE GARGALO (O INIMIGO)



COMPLEXIDADE
 $O(L^2)$
(QUADRÁTICA)

Se duplicares o tamanho do histórico (L), a memória exigida multiplica por 4.
O Transformer "rebenta" com Lags muito longos!

OS 5 TITÃS DO LONGO PRAZO (LSTF)

1

INFORMER



Truque: ProbSparse Attention (Lê apenas as ligações dominantes).

Ganho: Baixa a memória para $O(L \log L)$.
Prevê tudo de rajada (Single Forward Step), sem acumular erro passo-a-passo.

2

AUTOFORMER



Truque: Despede a Attention tradicional!

Usa Auto-Correlation (via FFT - Frequências de Fourier).

Ganho: Faz Decomposição de Série (Trend + Seasonality) nativamente dentro das suas camadas.

3

FEDformer



Truque: Opera puramente no Domínio da Frequência (Fourier / Wavelets).

Ganho: Complexidade estritamente Linear $O(N)$.
Usa Mixture of Experts (MOE) para extrair tendências brutais.

4

PatchTST



Truque 1: Patching → Corta a série em blocos. Em vez de ler 100 pontos, lê 10 "patches". Abate a barreira do $O(L^2)$.

Truque 2: Channel-Independence → Lê cada variável isolada no backbone. Zero mistura de ruído entre canais!

5

TFT
(TEMPORAL FUSION TRANSFORMER)



Foco: Explicabilidade (Não é uma caixa negra).

Truque: Usa Variable Selection Networks (VSN) para dizer que variável importa, e Gated Residual Networks (GRN) para "desligar" variáveis inúteis.

Output: Cospe incerteza nativa (P10, P50, P90) direto da rede!



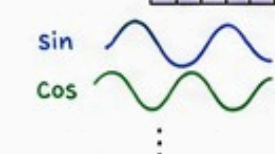
CAIXA DE
ALERTA
GIGANTE



AS DUAS PEÇAS OBRIGATÓRIAS DO TRANSFORMER:

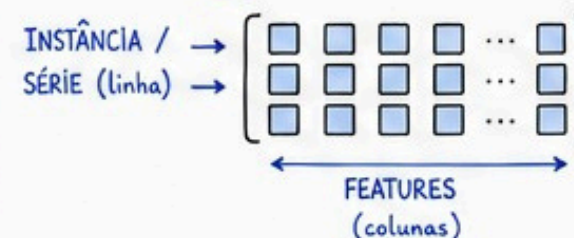
① POSITIONAL ENCODING (SENOS E COSSENOS)

TEMPO → 0 1 2 3 ... L-1



Sem isto, a rede não tem noção do tempo. Trata a série como um "saco de números desordenados".

② LAYER NORMALIZATION (vs BATCH NORM)



Transformers em séries temporais usam Layer Norm porque normaliza por instância (linha a linha), lidando perfeitamente com sequências de tamanhos variáveis.